The Agentic Decoherency Tumbler Problem

How Repeated Machine-Generated Code Modifications Erode Structural Coherence in Mature Software Systems

Registry: [HOWL-LLM-7-2026]

Series Path: [HOWL-LLM-2-2026] → [HOWL-LLM-3-2026] → [HOWL-LLM-4-2026] → [HOWL-LLM-5-2026] → [HOWL-LLM-7-2026]

DOI: 10.5281/zenodo.20431667

Date: May 2026

Domain: Human-LLM Interaction Methodology, Software Engineering

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

I. ABSTRACT

Large language models are increasingly deployed as autonomous code-generation agents operating on mature production codebases. This paper identifies a structural degradation process — the agentic decoherency tumbler — in which repeated machine-generated modifications cumulatively erode the internal coherence of a software system while leaving standard quality metrics undisturbed.

The mechanism operates as follows. An agentic session begins from a default state that does not include the system's full constraint web — the accumulated interdependent decisions that define why the code is shaped the way it is. Without iterative alignment between the model's statistical defaults and the system's specific structural requirements, each generated modification drifts toward the median of the model's training distribution. That drift is small on any single pass. But each modified codebase becomes the input for subsequent agentic sessions, shifting the local statistical center and increasing the probability of further drift on the next pass. The process is self-accelerating: the tumbler's output feeds back as its input.

The resulting degradation has several components: convergence of structurally diverse code toward homogeneous median patterns, loss of local optimizations that reflected problem-domain asymmetries, expansion of code volume without corresponding expansion of capability, accumulation of implementation decisions with no provenance or rationale, and displacement of the human constraint-web knowledge required to detect or reverse the process.

This degradation is invisible to the metrics that drive organizational decisions — velocity, test pass rates, headcount efficiency — and becomes visible only through lagging indicators that appear after the structural damage is done. The process is irreversible by the

same mechanism that produced it, because the agent is the tumbler.

No claims are made about LLM cognition or intent. The problem is entirely mechanical, arising from the interaction between how token prediction works, how context windows constrain what the model can observe, and how iterative application of a median-seeking process to a feedback loop produces convergent degradation.

II. HOW LARGE LANGUAGE MODELS GENERATE CODE

A large language model produces output by predicting the next token — the next word or word-fragment — based on two inputs: the statistical patterns encoded in its training weights and the content currently present in its context window.

The training weights are fixed. They encode the patterns of the model's training corpus — billions of lines of code, documentation, discussions, and text from across the internet. These weights define the probability landscape the model navigates: given what has come before, what is most likely to come next.

The context window is the live input. It contains the system prompt, the user's instructions, any loaded documents, and the model's own prior outputs within the current session. The context window selects which regions of the training weights activate for each prediction. A context full of Python activates Python patterns. A context full of database schemas activates data modeling patterns.

The critical point for this paper is what "most likely to come next" means in practice. The model does not select the best token, the most efficient token, or the most architecturally appropriate token. It selects the most probable token given the combination of training weights and current context. Most probable means the statistical center of the training distribution for that situation — the median pattern, the common approach, the way most code in the training data handled a similar context.

This is not a flaw. It is the mechanism. Every behavior described in this paper follows from this mechanism operating as designed.

III. THE CONTEXT WINDOW AS WORKING MEDIUM

The context window — the space containing all inputs and outputs for a single session — has four properties that constrain everything an LLM can do within a session.

It is **append-only**. Every token entered by the user or generated by the model is permanent for the life of the session. A misloaded document on turn one remains in the context at turn fifty. An early misinterpretation influences every subsequent prediction whether or not it was corrected.

It is **finite**. Current context windows range from roughly 200,000 to 1,000,000 tokens depending on the model and configuration. When the limit is reached, the session termi-

nates or earlier content is silently dropped. Every token committed to the window is an irreversible expenditure of a bounded resource.

It is **partially uncontrolled**. The user controls their inputs. They do not control the model's outputs — their length, their content, or their structural assumptions. A request for a focused 200-token response may produce a 3,000-token response containing tangents and unnecessary abstractions. That entire output is now permanent context influencing all subsequent predictions.

It is **cumulative**. Each prediction is influenced by the full context. Content from the first turn can influence predictions on the fiftieth turn. The quality of output at any point in a session is not a property of the most recent prompt — it is a property of the entire accumulated context from the session's first token to the current prediction.

These properties mean that what you load into the context, when you load it, and how the model first engages with it determine the trajectory of the entire session. A well-managed context produces increasingly constrained, on-target output. A poorly managed context accumulates noise that degrades every subsequent prediction.

IV. THE DEFAULT START PROBLEM

Every LLM session begins from a starting state. For an agentic coding session — where an automated system is tasked with making a code change — that starting state is one of two configurations.

Training weights only. The agent receives a task description and generates code from the training distribution with no project-specific context. The output reflects the statistical median of how all code in the training corpus handled similar tasks. It knows nothing about the specific system it is modifying.

Training weights plus a codebase snapshot. The agent loads some portion of the project's code into the context window before generating. This is better — the context now activates project-specific patterns from the training weights. But the snapshot is flat. It contains the current state of the files without the history of why they are shaped the way they are. It is a photograph of a living system taken at one moment, showing positions without showing forces.

Neither starting state includes the constraint web — the full set of interdependent decisions, implicit contracts, behavioral expectations, and structural rationale that define a mature codebase. Neither includes the judgment of why a particular function is shaped unusually, why a specific module is deliberately asymmetric, why a seemingly inefficient pattern was chosen over a seemingly efficient one. That knowledge exists in the accumulated experience of the people who built and maintained the system over years. It is not in the files. It cannot be loaded.

The agent begins every session with a partial, stale, flat view of a deep, live, interconnected system. What it does with that view depends on what happens next — or what fails to happen next.

V. THE ALIGNMENT GAP

When a human works with an LLM interactively on a complex problem, a critical phase occurs between loading information and producing output. The human iteratively corrects the model's initial responses — which default to generic templates drawn from the training distribution — until the model's predictions track the actual structure of the specific problem rather than the statistical template for "problems that look like this."

This phase is called alignment. It is where the model's default trajectory, shaped by training weights pointing at the median of all code, gets redirected toward the specific structural requirements of the system at hand. Alignment costs turns and tokens. It requires a human who understands the target structure well enough to recognize when the model is tracking it versus when the model is producing plausible-looking output from its default distribution.

Agentic workflows skip this phase. The agent receives a task description, loads whatever context is available, and proceeds directly to code generation. There is no iterative correction. There is no human evaluating whether the model's engagement with the codebase is tracking the system's actual structural logic or merely pattern-matching to the training distribution's default for "code that looks like this." There is no redirection from the median toward the specific.

The result is an alignment gap — a delta between the training distribution's median patterns and the system's actual structural requirements. This gap is present in every line the agent generates. It may be small on any given line. It is present on all of them. The training weights are pointing at how most code works. The system needs how this specific code works. Without alignment, the agent produces the former when the system needs the latter.

VI. THE CONSTRAINT WEB

A mature production codebase is not a collection of files. It is a web of interdependent decisions accumulated over years by many people working under many different conditions toward many different immediate goals.

Some of these decisions are explicit. A function signature is an explicit contract. A database schema is an explicit structure. A test is an explicit behavioral specification.

Most are implicit. Module A processes records in a specific order because Module B downstream assumes that order. A function is deliberately inefficient because the efficient version caused a race condition under production load three years ago and the fix was to serialize the operation. An interface is deliberately sparse — missing methods that seem like they should exist — because adding them would create a coupling between subsystems that must remain independent. A configuration value is set to a specific number because

a customer integration breaks at other values and no test covers this because the failure only manifests at scale.

These implicit decisions are the constraint web. Each one is load-bearing. Each one was made for a reason. Many of those reasons are not documented anywhere — not in comments, not in commit messages, not in design documents. They exist in the memory of the person who made the decision, or in the institutional knowledge of a team, or nowhere at all except as the structural shape of the code itself.

The constraint web of a large production system — 150,000 lines serving real customers — may contain tens of thousands of such interdependent decisions. The web is not static. Each deployment, each customer behavior change, each infrastructure update, each new feature adds new constraints and modifies existing ones. The web is a live, evolving, high-dimensional structure.

Tests cover a fraction of it. Documentation covers less. The fraction that is explicitly specified anywhere is small relative to the whole.

VII. WHY AGENTS CANNOT SEE THE CONSTRAINT WEB

The constraint web cannot fit in a context window.

Consider the arithmetic. A 150,000-line codebase is roughly 500,000 to 600,000 tokens depending on language and density. A 1,000,000-token context window — the upper end of current availability — can hold the raw code, but doing so consumes over half the session budget before any work begins.

The code alone is insufficient. The system's operational environment — deployment configurations, infrastructure definitions, CI/CD pipelines, service dependencies, environment variables — is another significant token expenditure. Documentation that explains how the system actually works, as opposed to API reference documentation, adds more. Production logs showing actual runtime behavior add more still.

Even if all of this could be loaded — and in practice it cannot, because the budget would be entirely consumed — the constraint web would still be absent. The constraint web is not the code. It is the history of decisions that produced the code and the reasons those decisions were made. A function's current implementation can be loaded. The knowledge that it was rewritten three times because the first two versions caused production incidents under specific conditions cannot be loaded, because that knowledge is not a file. It is the accumulated judgment of the people who lived through those incidents.

The agent operates from what fits in the window. The constraint web exceeds the window. Therefore the agent operates without the constraint web. Every decision the agent makes — every pattern it selects, every abstraction it introduces, every optimization it applies or removes — is made without visibility into the full set of constraints that decision must satisfy.

VIII. THE SINGLE-PASS PROBLEM

Consider one agentic modification to a mature codebase. The agent loads relevant files, receives a task description, generates a change, and the change passes all tests.

What happened during generation is mechanically straightforward. The agent's training weights contain the statistical distribution of all code patterns in the training corpus. The context window contains the loaded files. The agent predicted the most probable code for the task given those two inputs. The most probable code is the median of the training distribution as conditioned by the local context.

If the existing code matches the training distribution's median — if it was written the way most code handles this situation — the agent's output will be structurally consistent with the existing code. The change integrates smoothly. No damage occurs.

If the existing code deviates from the median — if it was written in an unusual way for a specific reason — the agent faces a prediction choice. The loaded code pulls toward the existing pattern. The training weights pull toward the median. The resolution depends on the relative strength of these signals, but the training weights represent billions of examples and the local context represents one codebase. Over a large enough number of predictions, the training distribution's gravity wins.

The result on a single pass is subtle. An unusual-but-correct optimization gets replaced with a common-but-generic implementation. A deliberately sparse interface gets "improved" with defensive null checks that the original author intentionally omitted because null is not a valid state in that context. A carefully structured data flow gets rearranged into the more common pattern that the training distribution favors, breaking an implicit ordering dependency that no test covers.

Each of these changes is locally reasonable. Each passes tests. Each would survive a review focused on "does this do what the task description asked." None of them would survive a review by someone who knew why the original code was shaped the way it was.

IX. THE TUMBLER EFFECT

A rock tumbler is a simple machine. Stones go in with distinct shapes — edges, facets, asymmetries formed by their specific geological history. The tumbler rotates. On each pass, the stones grind against the abrasive medium and against each other. Each pass removes a small amount of material. No single pass is destructive. After enough passes, every stone is smooth, rounded, and nearly identical. The specific features that made each stone distinct are gone.

The agentic decoherency tumbler operates by the same mechanism on code.

Pass one: the agent modifies the codebase. Some specific structural features are smoothed toward the median. The modified code is committed. The codebase has changed.

Pass two: the agent loads the modified codebase. The local context now includes the previous pass's output. The statistical center of the loaded context has shifted slightly

toward the training distribution's median — because that is what the previous pass introduced. Patterns that were already unusual are now slightly more unusual relative to their local context. The agent's predictions are slightly more likely to "correct" them toward the median. A second smoothing pass occurs.

Pass three through N: the process continues. Each pass shifts the local statistical center. Each shift makes the remaining specific structural features more anomalous relative to their surroundings. Each anomaly is more likely to be smoothed on the next pass. The tumbler accelerates its own effect.

This is the feedback loop that makes the process self-accelerating rather than merely linear. If each pass operated on the original codebase independently, the drift would be constant per pass. Because each pass operates on the output of all previous passes, the drift compounds. The tumbler's input is its own output.

The result, over enough passes, is convergence. Structurally diverse code — modules with different shapes reflecting different problem domains, subsystems with intentional asymmetries, components with specific optimizations for their specific contexts — converges toward homogeneous median code. The same patterns everywhere. The same abstractions everywhere. The same defensive checks everywhere. Consistency achieved through the loss of specificity.

This convergence looks like improvement on a surface read. The code looks cleaner, more consistent, more "modern." It follows current best practices as represented in the training data. It is easier for the next agent to process because it is closer to the patterns the agent already expects. The tumbler has made the code more legible to the tumbler. It has made the code less faithful to the problem domain.

X. CODE EXPANSION

Large language models do not minimize solutions. They produce the most probable solution, and the most probable solution in the training distribution is verbose.

This is because the training corpus reflects the full range of human coding practice, weighted by volume. Defensive programming patterns, redundant null checks, extra abstraction layers, verbose error handling, wrapper functions that add indirection without adding functionality — these are common in the training data because they are common in real-world code. They are common in real-world code for many reasons: corporate coding standards, inexperienced contributors, defensive habits from unreliable environments, copy-paste inheritance from templates and Stack Overflow answers.

The median of this distribution is not bad code. It is adequate code with more structure than any given problem requires. Applied once, the overhead is minor. Applied thousands of times across thousands of agentic modifications, the overhead accumulates.

A codebase that was 150,000 lines under human maintenance grows. Not because new features were added, but because each agentic pass adds slightly more code than the change strictly requires. An extra abstraction layer here. A redundant validation there.

A utility function that wraps a standard library call for no clear reason. Each addition is individually defensible. Collectively, the code-to-function ratio degrades.

This expansion is itself a form of decoherency. The relationship between the code's volume and its function — the density of the solution — loosens. More code does the same work. The signal-to-noise ratio of the codebase drops. Finding the code that matters — the code that implements the actual logic as opposed to the code that wraps, checks, validates, and abstracts around it — becomes harder. Understanding the system requires reading more code to extract the same information.

The expansion also makes the codebase harder for the agent itself to process in future sessions. More code means more tokens consumed during loading, which means less budget for alignment and generation. The tumbler is not only smoothing the code — it is expanding it, consuming more of the finite context window on each subsequent pass, and reducing the model's working budget for the changes that matter.

XI. PROVENANCE LOSS

When a human engineer makes an implementation decision — choosing one pattern over another, structuring a module a particular way, adding or omitting a specific check — the reasoning exists somewhere. It may be in a commit message. It may be in a pull request discussion. It may be in a design document. It may only be in the engineer's memory. But it exists. Someone chose, and the choice was made for a reason, and that reason is in principle recoverable.

When an agent makes an implementation decision, there is no reasoning. There is prediction. The agent did not choose a pattern because it was the right one for this context. It produced the most probable pattern given its training distribution and the loaded context. There was no evaluation of alternatives. There was no consideration of constraints. There was no decision.

The output looks identical to human-authored code. It has the same syntax, the same structure, the same naming conventions. It passes the same tests. In a code review, it is indistinguishable from a competent human implementation. The difference is invisible: behind the human implementation is a rationale; behind the agent implementation is a probability distribution.

As agentic modifications accumulate, the codebase fills with implementation choices that have no provenance. No one can explain why a particular pattern was used in a particular place, because no one chose it. The constraint web grows — every new line of code is a new node — but it grows opaquely. The new nodes have no edges to a rationale. They connect to the existing web only through syntactic adjacency, not through reasoned dependency.

This makes the next modification — whether human or agentic — more dangerous. Understanding the cost of a proposed change requires understanding why the current code is shaped the way it is. If the current shape is the result of reasoned decisions, those reasons

can be investigated. If the current shape is the result of statistical prediction, there is nothing to investigate. The question "why is it this way" has no answer, which means the question "what breaks if I change it" has no reliable answer either.

XII. THE REVIEW COLLAPSE

The organizational model accompanying agentic coding typically reassigns engineers from authoring code to supervising agent output. The engineer becomes a PM-style controller: they describe the desired change, the agent produces it, and the engineer reviews and approves.

This model changes the review function in two ways that compound the tumbler effect.

First, the review scope narrows. The engineer-as-author reviewed their own work against their knowledge of the local constraint web — the dependencies, the implicit contracts, the reasons for existing structural decisions in the code they worked in daily. The engineer-as-controller reviews agent output against the task description — does this change do what I asked for. The first review catches constraint web violations. The second does not, because it is not asking about constraints. It is asking about requirements.

Second, the review breadth expands. The engineer-as-author worked in a specific part of the codebase and developed deep knowledge of that area's constraint web. The engineer-as-controller can direct the agent to modify any part of the codebase, because the agent can generate code for any context. The controller reviews changes in areas where they have no constraint web knowledge — where they cannot catch violations because they do not know what the constraints are.

The result is that the review layer thins precisely as the need for constraint-aware review deepens. The tumbler is running, making changes that are locally reasonable but structurally erosive, and the review function that could catch the erosion has been reorganized away from the specific domain knowledge that made it effective.

The old model — specialist engineers reviewing changes in their area of expertise — was slow. It was slow because the engineers were evaluating changes against a large, partially implicit constraint web, and that evaluation takes time. The new model is fast because that evaluation has been removed. The speed is not free. It is purchased by accepting unreviewed constraint web violations on every pass.

XIII. THE METRICS TRAP

Every form of degradation described in this paper is invisible on the metrics that drive organizational decisions about adopting and expanding agentic coding workflows.

Velocity increases. More changes ship per engineer per sprint. The agent produces code faster than a human. The PM-controller can supervise multiple concurrent changes. The throughput numbers are real.

Test pass rates remain stable. The tumbler does not produce code that fails tests. It produces code that passes tests while violating constraints that tests do not cover. Test suites are a subset of the constraint web. Stable test pass rates indicate that the tested subset is intact, not that the full web is intact.

Headcount efficiency improves. Fewer engineers produce more changes. The cost per feature drops. The financial case is clear and immediate.

Code quality metrics may improve. Linting scores go up because the agent produces code that follows common conventions. Complexity metrics may drop because the agent's median code avoids the unusual patterns that complexity analyzers flag. The tumbler's smoothing effect looks like quality improvement to tools that measure convention adherence.

The degradation appears on different metrics, and it appears later.

Time-to-debug increases. When a production issue occurs, diagnosing it requires understanding why the code is shaped the way it is. Code with no provenance resists diagnosis. Code that was smoothed away from its problem-domain-specific structure produces failures that do not map cleanly to the code's current organization.

Regression rates on cross-cutting changes increase. Modifications that touch multiple subsystems encounter more invisible constraint violations because more constraints have been obscured by the tumbler.

Fragility under load increases. Performance characteristics that depended on specific implementation choices — choices the tumbler smoothed away — degrade under production conditions that tests do not replicate.

Modification difficulty increases. The expanding codebase requires more reading to understand. The smoothed structure provides less information per line about the problem domain. The absent provenance means each modification requires more investigation to determine what is safe to change.

These lagging indicators appear months or years after the process that caused them. They appear after the velocity metrics have been reported, after the headcount decisions have been made, after the specialist engineers have been reassigned or departed. They appear as "technical debt" — the industry's generic term for structural degradation whose specific causes have been forgotten or were never identified.

XIV. IRREVERSIBILITY

A tumbled stone cannot be untumbled. The material removed by each pass is gone. The original geometry — the specific edges, facets, and asymmetries formed by geological processes over millions of years — cannot be reconstructed from the smooth result.

The agentic decoherency tumbler produces the same irreversibility in code.

Once a specific structural decision has been overwritten by a median pattern, the knowledge of what the original decision was and why it was made is gone from the codebase. If that knowledge also exists in a human's memory or in archived documentation, recovery is possible in principle. If the humans who held that knowledge have been reassigned to PM-controller roles where they no longer maintain it, or have left the organization, the knowledge is gone entirely.

The natural response — "use the agent to refactor back to a coherent state" — fails for the reason that defines the problem. The agent is the tumbler. It will refactor toward the training distribution's median, which is the same direction it was already going. Pointing the tumbler at its own output to undo its own effect produces more of the same effect. The agent cannot restore structural decisions it has no representation of. It cannot optimize toward a target architecture that exists only in the judgment of people who are no longer in the loop.

Recovery from advanced tumbler degradation requires humans who hold or can reconstruct the constraint web, working manually to re-establish the specific structural decisions the system needs. This is the most expensive form of software engineering work. It is the work that the agentic workflow was adopted to avoid. The process of avoiding it creates the conditions that make it necessary.

XV. FALSIFICATION CRITERIA

F1 — Tumbling is detectable and preventable by tooling. If static analysis, architectural fitness functions, constraint-specification tools, or other automated mechanisms can detect and prevent single-pass structural drift at scale — without requiring human constraint web knowledge in the review loop — then the tumbler effect can be arrested mechanically. This would not invalidate the description of the mechanism but would limit its practical impact to organizations that fail to adopt available tooling.

F2 — Agentic output does not converge on the median. If measured across large codebases over time, agent-generated code maintains the same structural diversity and local optimization as human-authored code, the convergence claim is wrong. Testable by comparing structural metrics — pattern diversity, abstraction density, code-to-function ratio — between agentic and human-maintained codebases performing equivalent feature work over equivalent time periods.

F3 — Expanded code is not less maintainable. If codebases that grew through agentic expansion show equivalent or better maintainability metrics — time-to-modify, regression rates, debug time, onboarding time for new contributors — compared to human-maintained baselines of equivalent functionality, then code expansion is not a meaningful form of decoherency.

F4 — PM-controller review is sufficient. If engineers operating in PM-controller roles catch constraint web violations at rates comparable to domain-specialist reviewers working in their area of expertise, the review collapse claim is wrong. Testable by controlled

comparison of review effectiveness across organizational models on equivalent change sets in equivalent codebases.

F5 — Future models hold the full constraint web. If future model architectures with sufficient context, persistent memory, or retrieval mechanisms can maintain an accurate, current, complete model of a mature system's full constraint web — including the rationale behind structural decisions — and can evaluate proposed changes against it, then the problem described here is a capability gap in current systems rather than a structural limitation of the approach. This would not invalidate the mechanism for current systems but would limit the paper's longevity.

F6 — Codebases recover through continued agentic work. If agentic refactoring passes, without human constraint web knowledge in the loop, can measurably restore structural coherence to a codebase that has undergone significant tumbler degradation, then irreversibility is wrong and the tumbler is a reversible process that self-corrects given sufficient passes. Testable by measuring structural metrics before, during, and after sustained agentic refactoring of a degraded codebase.

F7 — The effect does not compound. If measured degradation per agentic pass remains constant over time rather than accelerating — if each pass degrades the codebase by the same amount regardless of how many prior passes have occurred — then the feedback loop does not exist as described. The reality would be linear drift rather than self-accelerating decoherency, and the tumbler metaphor overstates the severity. Testable by measuring per-pass structural drift rates across many sequential agentic modifications to the same codebase.

XVI. CONCLUSION

The agentic decoherency tumbler is running. It is running in every organization that has adopted agentic coding workflows against mature production codebases. It is running because the mechanism is inherent in how the tools work — not in how they fail, but in how they succeed.

Large language models generate the most probable code. The most probable code is the statistical median of the training distribution. Applying median-seeking predictions to a codebase whose value lies in specific, non-median structural decisions erodes those decisions. Feeding the output back as input for the next pass accelerates the erosion. Removing domain-specialist review from the loop removes the function that would catch the erosion. Measuring velocity instead of structural coherence hides the erosion. Displacing the humans who hold constraint web knowledge makes the erosion irreversible.

No single step in this process is a mistake. Each step follows rationally from the incentives and information available to the decision-makers. The agent produces code that passes tests — use it. The PM-controller model improves throughput — adopt it. The velocity metrics are positive — expand the program. The specialist engineers are underutilized in the new model — reassign them. Each decision is locally reasonable. The cumulative result is a codebase that is progressively less faithful to its problem domain, progressively

harder to maintain, and progressively more expensive to recover — if recovery is possible at all.

The tumbler cannot be stopped by building a better tumbler. It can be stopped by understanding what it does and choosing where to run it. Greenfield code with no constraint web, small systems that fit entirely in context, well-specified tasks with explicit constraints — these are domains where agentic coding works with the mechanism rather than against it. Mature systems with deep implicit constraint webs, accumulated structural decisions, and live customer dependencies — these are domains where the tumbler runs at the highest speed and the damage is least visible until it is least reversible.

The skill is not in using the tools. The skill is in knowing where the tools work and where the tools are the problem.

[**HOWL-LLM-7-2026**] The Agentic Decoherency Tumbler Problem. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-7-2026

[**HOWL-LLM-2-2026**] Calibrate, Extract, Refine. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/H
LLM-2-2026

[**HOWL-LLM-3-2026**] Riding the Rocket. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-3-2026

[**HOWL-LLM-4-2026**] Incompatibility by Construction. Github: https://github.com/ghowland/papers/tree/main/papers/L
LLM-4-2026

[**HOWL-LLM-5-2026**] What I Cannot Do. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-5-2026